Multi-Agent AI Workflow Engine

Simulating an Autonomous Software Development Team

Marco Perozo • Gabriel Marte • Lewdy Aquino • Alvin Almonte

Department of Computer Science · Southwest Minnesota State University · Capstone Project, Fall 2026

Introduction & Problem Statement

Large Language Models (LLMs) can now generate source code from natural-language descriptions, and tools such as GitHub Copilot and ChatGPT are used daily by developers (Chen et al., 2021). Almost all of them are built around one model receiving one prompt: a single pass in which the model must understand the request, design the solution, write the code and verify it all at once.

Three limitations follow:

- Single-model limits. One pass over a large, multi-step task produces incomplete or incorrect results on anything non-trivial.
- Lack of teamwork. Real software needs a product manager to clarify, a developer to build and QA to verify. A single prompt collapses these perspectives and loses the checks and balances that make teams effective.
- Lack of iteration. Standard prompts do not reliably self-correct against predefined requirements (Madaan et al., 2023); detecting and fixing errors falls to the human.

RESEARCH QUESTION

Does a structured, role-based multi-agent workflow (PM → Developer → Reviewer with an iteration loop) produce more correct and more requirement-compliant software than a single LLM given the same task — and at what cost in tokens and time?

Goal & Objectives

Goal. Design, implement and evaluate an engine that simulates an autonomous software development team and turns a simple prompt into working, reviewed code through a controlled iteration loop.

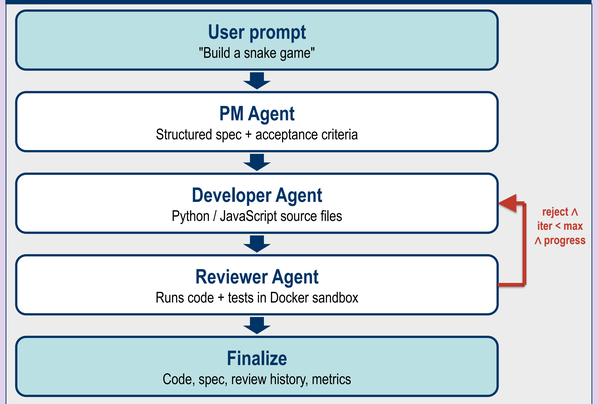
- Build the orchestration pipeline as a LangGraph state graph.
- Implement PM, Developer and Reviewer agents, each producing a well-defined artifact (spec, code, review report).
- Engineer loop prevention so the review/fix cycle always terminates.
- Implement memory management (rolling summarization and pruning).
- Persist every run, message, spec, code version, review and metric.
- Evaluate against a single-model baseline on ≥ 20 hidden-test tasks.

Related Work

Agentic frameworks show that decomposing a problem across cooperating LLMs improves quality on complex tasks: AutoGen provides a general multi-agent conversation layer (Wu et al., 2024); MetaGPT encodes standard operating procedures for software roles (Hong et al., 2024); ChatDev organises agents into a virtual software company (Qian et al., 2024). Self-Refine (Madaan et al., 2023) and Reflexion (Shinn et al., 2023) show that iterative self-feedback raises output quality, and SWE-bench (Jimenez et al., 2024) supplies a rigorous hidden-test evaluation protocol.

Gap addressed. These systems report capability, but rarely report guaranteed termination or cost per successful task. MAWE makes the control flow explicit and testable, and measures cost-per-success directly.

System Architecture



Six layers. Orchestration (LangGraph state graph) · Agent (role prompts and tools) · Safety (iteration caps, exit conditions, fallback routing) · Memory (summarization and pruning) · Persistence (SQLAlchemy / SQLite → PostgreSQL) · Evaluation (benchmarks against a single-model baseline).

Methodology

Development process. Four two-week Agile sprints; every change goes through a GitHub pull request reviewed by a teammate; GitHub Actions runs the unit-test suite on each push; 15-minute weekly stand-ups on a Kanban board.

Workflow as a state graph. Nodes are agent steps (plan, develop, review, summarize, finalize); edges are routing rules evaluated on the current state, which makes the control flow explicit, testable and easy to visualise.

START → PM.plan → Dev.implement → Reviewer.review → [approve → finalize] | [reject ∧ iter < max ∧ progress → Dev.implement] | [reject ∧ (iter = max ∨ no progress) → finalize_with_warnings] → END

Testing strategy. Unit tests for routing, loop-prevention rules, summarization and database access; integration tests over the full pipeline with a mocked LLM for determinism; end-to-end runs with real LLM calls before each milestone.

Loop Prevention & Memory Management

The two failure modes that break autonomous agent pipelines are non-termination and context growth. Both are engineered against explicitly.

Four termination mechanisms

- Hard iteration cap — configurable, default 5.
- Severity threshold — only critical and major issues cause rejection; minor issues are reported but do not block approval.
- No-progress detection — a SHA-256 hash of the Developer output exits the loop when the same code or the same issue list repeats.
- Fallback routing — on exit without approval the engine finalizes the best version with a warnings report rather than failing.

Memory strategy

Only the latest spec, latest code and latest review stay in full context; older iterations are compressed into a compact progress log stored in memory_summaries, while every raw message is still persisted in the database so nothing is lost for later analysis.

Database Design

The database is the system's memory of record: every run is fully traceable from the original prompt through each code version to the final metrics. The schema is in third normal form, written with SQLAlchemy so SQLite (development) migrates to PostgreSQL unchanged.

Table	Purpose
users / projects	Owner and the original idea prompt
agents	Role, model, system prompt, version
runs	Mode, status, iterations, tokens, cost
messages	Every agent-to-agent exchange
specifications	Versioned PM spec + acceptance criteria
code_artifacts	Each file per iteration + SHA-256 hash
reviews	Verdict, severity, tests passed/failed
memory_summaries	Compressed context, tokens before/after
benchmark_tasks	Prompt, difficulty, hidden tests
evaluation_results	Metric name and value per run

USER ——< PROJECT ——< RUN ——< { MESSAGE >—— AGENT, SPECIFICATION, CODE_ARTIFACT, REVIEW, MEMORY_SUMMARY, EVALUATION_RESULT } · BENCHMARK_TASK ——< RUN

Implementation Status

Stack. Python 3.11+ · LangGraph · Anthropic / OpenAI API SDKs · subprocess sandbox with a wall-clock timeout, path containment and output truncation · pytest · SQLAlchemy over SQLite → PostgreSQL · FastAPI · Git and GitHub Actions. No GPU is required.

Docker with no network was the stated hardening target and is not implemented. Every execution goes through the subprocess path, which is a resource limit and a containment boundary rather than a security boundary.

Phase	Weeks	Milestone	Status
1. Setup & prototyping	1–2	M1 pipeline running	Met
2. Role implementation	3–5	M2 full PM/Dev/QA loop	Met
3. Memory & state	6–8	M3 engine v1.0	Met
4. Testing & evaluation	9–12	M4 benchmark + report	Met

Experimental Evaluation

Design. The same benchmark tasks are solved under four conditions using the same underlying model and temperature, so that any difference is attributable to the workflow rather than the model.

- A — Single model (baseline): one call, one answer.
- B — Single model with self-refinement: the same model writes, then reviews and fixes its own code under the same iteration cap.
- C — MAWE (proposed): PM → Dev → Reviewer with loop prevention and memory management.
- C' — Ablation: MAWE with summarization disabled.

Benchmark. ≥ 20 Python and JavaScript tasks with hidden tests the agents never see — 7 easy, 8 medium, 5 hard. Three repetitions per task per condition (240 runs) to account for LLM non-determinism; means reported with 95% confidence intervals. The test follows the metric: Wilcoxon signed-rank for tokens and pass rate, and McNemar's exact test for task success, which is binary and would otherwise be computed from a far smaller sample than it appears to use.

Metrics. Hidden-test pass rate, task success rate, specification coverage, iterations used, total tokens, estimated cost, latency, termination reason and context reduction from summarization.

Results

All four milestones met: 445 automated tests, green on Python 3.11, 3.12 and 3.13. The two hypotheses that need no language model are answered by measurement; the three that need one are not.

Hypothesis	Result
H4 every run terminates	Supported
H5 summarisation cuts context by 40%+	Supported
H1-H3 correctness, coverage, cost	Needs a model

H4. 100,000 randomised adversarial review sequences across four policy configurations: 0 failed to terminate and 0 exceeded a cap. H5. Context at 80 iterations falls from 52,494 to 3,316 tokens, a 93.7% reduction that never exceeds the budget, with C and C' solving the same tasks.

Pipeline check (fixture, NOT a model)	Solved	Pass rate
A — Single model	0 / 20	57%
B — Self-refinement	20 / 20	100%
C — MAWE	20 / 20	100%
C' — No summarization	20 / 20	100%

A fixture with known answers, not a model: it shows the comparison detects a difference it was built to detect, and says nothing about model ability.

Discussion & Conclusions

Settled. The loop provably terminates and the context stays bounded. The benchmark is validated as an instrument: every task is solvable, no task is satisfied by a do-nothing program, and injected defects are caught 91.4% of the time.

Not settled. Whether role separation produces more correct software, and whether it is worth the cost, need a sweep against a real model. The apparatus is complete; what is missing is an API key, not code.

Limitations. Scoring uses hidden tests rather than the Reviewer's verdict, since the Reviewer is itself an LLM; tasks are small to medium; results will depend on the model family; and C differs from B in two ways at once, role separation and grounded review, so a gain over B cannot be attributed to either alone.

References

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., … Zaremba, W. (2021). Evaluating large language models trained on code. arXiv:2107.03374.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., … Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. International Conference on Learning Representations (ICLR 2024).

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? International Conference on Learning Representations (ICLR 2024).

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., … Clark, P. (2023). Self-Refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems, 36.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., … Sun, M. (2024). ChatDev: Communicative agents for software development. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 15174–15186.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., … Wang, C. (2024). AutoGen: Enabling next-generation LLM applications via multi-agent conversation. Conference on Language Modeling (COLM 2024).

Acknowledgements

We thank Dr. Hezy for supervising this capstone and for feedback on the proposal and evaluation design, and the maintainers of LangGraph, whose tooling this work builds on.

POSTER TEMPLATE BY
www.PosterPresentations.com